PROJECT DOCUMENTATION & TECHNICAL WHITE PAPER

Braille Image Recognition & Translation Portal

An End-to-End Deep Learning & Computer Vision System for Automated Raised-Dot Decoding, Multi-Language Translation, AI Contextual Refinement, and Text-to-Speech Synthesis.

Document Class: Technical Whitepaper & System Architecture Guide
Target Domain: Optical Braille Character Recognition (OBCR) & Assistive Tech
Deployment Environment: FastAPI (Backend) & Next.js (React Frontend)
Date: May 2026
Version: 1.2.0 (Production-Ready)

1. Executive Summary

This project implements a state-of-the-art web portal designed to recognize, transcribe, translate, and verbalize double-sided printed Braille documents from raw photographic images. The system targets a key challenge in assistive technology: enabling sighted educators, parents, and transcribers to quickly digitize physical Braille pages. The portal incorporates advanced computer vision algorithms to pre-process images, a custom PyTorch Convolutional Neural Network (CNN) to classify Braille dots, a rule-based multi-language translation engine, a Generative AI layer for spelling/contextual correction, and text-to-speech synthesis.

2. System Architecture Overview

The system is divided into two distinct services communicating via JSON REST endpoints and Server-Sent Events (SSE) streaming:

- **Frontend Dashboard (Next.js & TypeScript):** Provides a high-performance web workspace. Users upload images, interactively select document corners for perspective correction, and adjust light angles using a visual dial. It handles SSE streaming inputs for real-time visualization of processing stages and includes text-to-speech playback controls.
- **Computational Backend (FastAPI & PyTorch):** A Python-based microservice that executes heavy image warping, bilateral filtering, double adaptive thresholding, belief propagation grid alignment, CNN character classification, dictionary translation, Google Gemini Generative AI corrections, and text-to-speech generation.

Component / Layer	Technologies Used	Primary Functionality
Client-Side UI	Next.js 16, React 19, TypeScript, Tailwind CSS, Lucide Icons, Framer Motion	Image workspace, interactive crop, real-time light visualizer, SSE streaming renderer, and TTS player.
Web Server	FastAPI, Uvicorn, Python Multiprocessing/ThreadPoolExecutor	Asynchronous endpoints, SSE progress stream, temporary file management, CORS middleware.
Computer Vision Pipeline	OpenCV, NumPy, SciPy (cKDTree), Numba (JIT)	Perspective warping, 4-pass tiled thresholding, wet-paper filtering, probabilistic layer splitting.
Machine Learning Decoder	PyTorch, TorchScript, CNN Model (3 channels)	3-channel patch extraction (32x32) and binary raised-dot state prediction (6-bit Braille configuration).
Translation & AI Layer	Custom Dictionaries, Google GenAI SDK (Gemini 2.5)	Multilingual decoding (AR, FR, EN, RU), language auto-detection, and contextual spelling refinement.
Speech Synthesis	Edge-TTS, Asyncio, Audio File streaming	Converts localized translation strings to high-fidelity audio streams for various dialects.

3. The Image Processing Pipeline (CV Layer)

A critical innovation of this project is the non-destructive physical feature extraction pipeline. Instead of running pure end-to-end OCR (which fails on physical Braille due to camera angles, page curves, and double-sided printing), the backend runs a multi-stage mathematical computer vision pipeline in **OpenCV** and **Numba**:

3.1. Perspective Warp & Curvature Rectification

Using 4 coordinates supplied interactively by the user, the backend performs a perspective warp (cv2.getPerspectiveTransform and cv2.warpPerspective) to extract the text sheet. This flattens page curl and guarantees a standardized orthographic coordinate workspace for grid alignment.

3.2. Light Angle Calibration & Self-Translation Vectors

Since Braille dots are 3D raised spheres, their visibility depends on shadows. The UI allows inputting the lighting angle. The backend uses double adaptive thresholding (highlight-seeking and shadow-seeking) to find highlighting and shadow pixels. By template matching highlight and shadow layers against each other (using template matching self-correlation), the system finds the exact displacement vector representing the dot radius and lighting direction.

3.3. Directional Wet-Paper (WB) Filtering

To filter out page noise and paper grain, a directional smoothing filter is implemented. It accumulates pixel differences along the primary horizontal, vertical, and diagonal directions. The accumulated 'wet-paper' gradient maps isolate raised dots while smoothing out flat areas and fibers.

3.4. Probabilistic Layer Splitting (Recto/Verso Separation)

Double-sided Braille documents (Interpoint) print characters on both sides of the sheet. Sighted readers or standard scanners see 'bleed-through' indentations from the back page (Verso) mixed with front-page dots (Recto). The system separates them mathematically: under directional light, front-page raised dots cast shadows in one direction, while back-page depressions cast shadows in the opposite direction. By analyzing pixel pairs shifted along the estimated light vector, the backend builds two independent probability layers: one representing Recto dots, and the other Verso dots.

Mathematical Formula:
Let C be the raw grayscale contrast. The shadow shifts v_1 and v_2 represent recto and verso displacements. The probability of recto dot presence is computed as:

$$P(Recto) = A / (A + B + \varepsilon)$$

Where A is the shifted intensity map under v_1, and B is under v_2.

4. Grid Discovery, Modulo Voting & Belief Propagation

Once noise-free dot pixels are separated, they must be grouped into structural 3x2 cells. This is solved via spatial alignment and voting filters:

4.1. Spatial Periodicity & Spacing Estimation

The system runs 2D template matching self-correlations on the binary dot masks to compute the structural rhythm of the page: the cell dot spacing (horizontal/vertical) and the inter-character/inter-line gaps.

4.2. Markov-style Belief Propagation (Modulo Voting)

Because of paper warping or hand-written variations, absolute grids fail. Instead, a sparse KD-Tree graph of dot centers is built. A belief propagation solver (using SciPy CSR sparse matrix operations) runs iterative voting: neighboring dots vote on whether they belong to the same cell row (Top, Middle, Down) or column (Left, Right) based on modulo distance properties. Unsure dot assignments are updated iteratively based on the consensus of their neighbors. This algorithm achieves high tolerance against page rotation and skew.

4.3. Grid Completion & Rescue

If some dots are missing due to low printing quality or scan wear, a grid density recovery step rescues them. It projects virtual cells horizontally and vertically and checks if loose pixels fit the pattern, 'rescuing' weak dots before decoding.

5. Convolutional Neural Network (CNN) Classifier

For each rescued 3x2 cell bounding box, the backend extracts a 3-channel image patch of size 32x32. The channels are structured to maximize feature density:

- **Channel 1 (Original Grayscale):** Represents raw texture information.
- **Channel 2 (Separated Recto Intensity):** Focuses purely on extracted front-page dots.
- **Channel 3 (Verso/Recto Contrast Boundary):** Emphasizes edge transitions.

These patches are passed to a PyTorch CNN model (DotPredictor) consisting of three Conv2D layers, Batch Normalization, Max Pooling, Adaptive Average Pooling, and fully connected layers. The network predicts the binary state (0 or 1) of the 6 dot positions in the cell, outputting a 6-bit binary sequence representing the Braille character.

```
class DotPredictor(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv_layers = nn.Sequential(
            nn.Conv2d(3, 32, kernel_size=3, padding=1),
            nn.BatchNorm2d(32), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.BatchNorm2d(64), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(64, 128, kernel_size=3, padding=1),
            nn.BatchNorm2d(128), nn.ReLU(),
            nn.AdaptiveAvgPool2d((1, 1))
        )
        self.fc_layers = nn.Sequential(
```

```
            nn.Linear(128, 64), nn.ReLU(),
            nn.Dropout(0.3), nn.Linear(64, 1), nn.Sigmoid()
        )
```

6. Multilingual Translation & LLM Refinement Layer

Once the CNN yields the Unicode Braille characters, the text is structured into reading lines by a custom **Universal Structural Assembler** (which preserves line feeds and spaces even on curved tables or margins). The text is then translated and processed through the following software stages:

6.1. Rule-Based Dictionary Translations

The translation engine supports five localized lookup tables based on standard international usage:

- **Arabic:** World Braille Usage standard (translating ligatures, harakat diacritics, and numeric indicator modes).
- **French:** Braille Français Uniformisé (BFU) standard (handling accented characters, capital indicator states).
- **English Grade 1:** UEB literal letter-for-letter mapping.
- **English Grade 2:** Unified English Braille contraction logic (handling whole-word and part-word rules).
- **Russian:** GOST R 58511-2019 Cyrillic mapping.

6.2. Auto-Language Detection

A highly powerful user option is *Auto-Detect*. The backend transcribes the Braille Unicode using all 5 translation tables simultaneously. It then compiles the 5 raw candidates into a single prompt sent to **Gemini 2.5 Flash Lite**. The model analyzes semantic coherence and returns a structured JSON package containing the identified language and a corrected translation, avoiding the need for manual selection.

6.3. Gemini AI Spelling and Grammar Correction

Because Braille images can contain worn paper or dust, small OCR typos can occur. The system leverages the Google GenAI SDK. If the selected language is not 'raw', the translation is sent to Gemini to fix punctuation, spacing, and formatting. The prompt is also tailored to identify fill-in-the-gap educational sheets, automatically replacing blank exam slots with clean dashes (-----).

6.4. Text-To-Speech (TTS)

The translation result is accessible to visually impaired or learning users through an integrated TTS engine powered by edge-tts. The backend queries available Microsoft Neural voices, categorizes them by language/dialect, and streams synthesized audio back to the frontend browser dynamically.

7. Streaming & Interactive Client Dashboard

To ensure a responsive user experience despite the latency of deep learning models and LLM API calls, the frontend and backend communicate via **Server-Sent Events (SSE)**. When processing an image, the FastAPI server yields chunks progressively:

1. **Stage 'overlay':** Streams the cropped page image with annotated character boxes immediately after grid completion.
2. **Stage 'braille':** Emits the transcribed raw Braille Unicode symbols.
3. **Stage 'detected_language':** (If in Auto-Detect mode) Identifies and emits the detected language key.
4. **Stage 'translation':** Emits the fast dictionary-translated text string.

5. **Stage 'gemini':** Emits the final context-corrected translation once the Gemini response arrives.

This progressive stream prevents the browser UI from locking up and keeps the user informed of the exact processing state.